Pediatric Clinical Database

Complete Concepts, Folder Guide, Important Queries and Viva Questions

PURPOSE	Learn the database as a system - what each concept means, where it is used, why it was selected, how to explain it, and how to answer viva questions confidently.

How to study this handbook

- First learn the table relationships and four form tables. These explain most of the project.
- Then learn constraints, normalization, indexes, transactions, soft deletion and audit history.
- Practise the important queries by explaining SELECT, JOIN, WHERE, GROUP BY and ORDER BY in that order.
- For every viva answer use this pattern: definition, project example, reason, and trade-off.

One-minute project answer

This is a MySQL 8 database-first pediatric clinical system. Relational tables store stable hospital data such as patients, visits, diagnoses, prescriptions and laboratory results. Primary keys identify rows, foreign keys maintain relationships, constraints protect quality, indexes improve common searches, transactions keep multi-table changes atomic, and audit/history tables preserve accountability. Dynamic clinical forms use relational template metadata plus validated JSON answers so hospitals can add fields without redesigning the whole schema. Clinical records use soft deletion so data is retained with the deletion time, user and reason.

01 Folder and file map

Know where every database deliverable lives and why it exists.

PATH	ROLE	WHY
database/migrations/001_schema.sql	Fresh database definition	Creates core tables, keys, constraints, indexes and JSON form tables.
database/migrations/002_views_routines.sql	Reusable database logic	Creates views, stored procedure and audit/safety triggers.
database/migrations/003_hospital_workflows.sql	Workflow extension	Adds identifiers, insurance and expanded follow-up structures.
database/migrations/004_json_dynamic_forms.sql	Legacy upgrade	Converts the old EAV form design into the JSON architecture.
database/migrations/005_soft_delete_and_retention.sql	Retention upgrade	Adds soft-delete metadata and changes clinical cascades to RESTRICT.
database/seeds/001_seed.sql	Demonstration data	Adds 30 synthetic patients, 7 doctors and connected clinical records.
database/seeds/002_followup_history.sql	Continuity demo	Adds older completed follow-ups and notes for comparison.
database/queries/evaluation_queries.sql	Marks/evaluation evidence	Contains joins, aggregates, subquery, window function, JSON query and transaction.
database/queries/performance_benchmarks.sql	Performance evidence	Shows EXPLAIN ANALYZE and keyset pagination patterns.
docs/DESIGN.md	Design explanation	Explains normalization, JSON choice, deletion and referential rules.
docs/EVALUATION_1.md	Evaluation guide	Maps first-evaluation requirements to database evidence.
server/src/routes/*.js	Database consumers	Shows how SQL is used safely by API routes; useful as implementation evidence.
client/src/pages/*.jsx	Demonstration UI	Not the marking focus, but proves the database features can be used.

Why separate migrations, seeds and queries?

Migrations define structure and controlled upgrades. Seeds create repeatable test data. Queries demonstrate learning and reports. Keeping them separate prevents demonstration data from becoming part of the schema and makes deployment order clear.

02 Database architecture and table groups

The design follows real clinical workflow: patient -> visit -> clinical facts -> follow-up.

DOMAIN	TABLES	RESPONSIBILITY
Security	users, roles, permissions, role_permissions	Who can act and which permissions a role has.
Patient identity	patients, guardians, patient_guardians, patient_identifiers	Demographics, contacts, legal relationships and external IDs.
Scheduling	appointments	Planned interaction before a visit.
Encounter	visits, medical_records, diagnoses, treatments	What happened clinically during one visit.
Measurements	vital_signs, growth_records	Time-sensitive observations and pediatric growth.
Safety/history	allergies, immunizations	Longitudinal facts attached to the patient.
Medication	medications, prescriptions, prescription_items	Drug catalogue, prescription header and multiple items.
Laboratory	lab_tests, lab_orders, lab_results	Test catalogue, order event and result.
Continuity	follow_ups, follow_up_notes	Tasks after a visit and their contact/activity history.
Dynamic forms	custom_forms, custom_form_fields, custom_form_responses, response_history	Versioned definitions, JSON answers and revisions.
Governance	audit_logs	Who changed what and when.

Central relationship

```
patients (1) ----< visits (1) ----< clinical records
     |                  |
     |                  +----< prescriptions ----< prescription_items
     |                  +----< lab_orders -------- lab_results
     |                  +----< follow_ups --------< follow_up_notes
     |                  +----< custom_form_responses ----< response_history
     +----< allergies / immunizations / growth history
```

03 Keys and relationships

CONCEPT	MEANING	WHERE USED	WHY
Primary key	Uniquely identifies one row.	patients.patient_id	Fast stable identity; names and phone numbers can change or repeat.
Surrogate key	Generated technical key with no business meaning.	BIGINT AUTO_INCREMENT	Simpler joins and stable references than using MRN as every foreign key.
Candidate/business key	Real-world value that should be unique.	patients.medical_record_number	Prevents duplicate hospital records while retaining a technical PK.
Foreign key	Requires referenced parent row to exist.	visits.patient_id -> patients.patient_id	Stops orphan visits.
Composite unique key	Uniqueness across multiple columns.	custom_forms(form_key, version)	Same form key may have versions, but a version cannot be duplicated.

Relationship examples

- One-to-many: one patient has many visits. The foreign key is on the many side: visits.patient_id.
- One-to-one: one visit has at most one medical record or one growth row because visit_id is UNIQUE in those tables.
- Many-to-many: patients and guardians use patient_guardians. The junction table stores relationship, primary-contact and custody facts.
- Header-detail: prescriptions is the header; prescription_items stores one or more medicines. This avoids repeating doctor/date for every item.

Why not use patient name as the primary key?

Names are not unique, may change, can contain spelling differences, and are larger/slower to join. A numeric surrogate key is stable; MRN is separately protected with UNIQUE.

04 Data types and constraints

TYPE	USED FOR	WHY
BIGINT UNSIGNED	IDs	Large positive range and common type on both sides of each FK.
VARCHAR(n)	Names, codes, short notes	Length limit documents intent and prevents uncontrolled text.
TEXT	Clinical narrative	Suitable for longer history/examination text.
DATE	Birth date, due date	A date without a time zone/time-of-day.
DATETIME	Visit and event time	Stores clinical date and time consistently.
TIMESTAMP	created_at / updated_at	Automatic CURRENT_TIMESTAMP behavior for row metadata.
DECIMAL	Height, weight, temperature	Exact decimal storage; avoids floating-point surprises.
BOOLEAN	Flags	Readable true/false intent; MySQL stores it as a tiny integer.
ENUM	Small controlled states	Rejects unknown states such as an invalid form status.
JSON	Form rules and answers	Flexible document with MySQL JSON validation and query operators.

Constraints are database-level validation

CONSTRAINT	PURPOSE	EXAMPLE
NOT NULL	Value is mandatory	patients.first_name
DEFAULT	Safe initial value	custom_forms.status defaults to draft
UNIQUE	No duplicate value/combination	medical_record_number; form_key + version
CHECK	Business rule must be true	version >= 1; JSON_TYPE(answers)='OBJECT'
FOREIGN KEY	Referenced parent must exist	response.visit_id references visits

Why validate in both API and database?

The API gives friendly messages. Database constraints are the final protection for every client, import script and administrator. One layer improves usability; the other preserves integrity.

05 Normalization and the JSON decision

Most stable clinical data is normalized to third normal form; variable form answers use a controlled hybrid design.

FORM	RULE	PROJECT EXAMPLE
1NF	Atomic values; no repeating columns	Prescription medicines are rows in prescription_items, not medicine1, medicine2, medicine3 columns.
2NF	Every non-key attribute depends on the whole key	In patient_guardians, relationship describes that patient-guardian link, not only the patient.
3NF	No non-key attribute depends on another non-key attribute	Medication name/strength live in medications; prescription_items keeps medication_id.

Why normalization?

- Reduces duplicate data and update anomalies.
- Makes one source of truth for medicines, users and lab tests.
- Improves referential integrity and reporting consistency.
- Trade-off: reports need JOINs, so indexes and views are important.

Why JSON for dynamic responses instead of pure EAV?

OPTION	ADVANTAGE	PROBLEM / DECISION
Fixed columns	Very easy SQL	Every new form field needs ALTER TABLE; unsuitable for dynamic forms.
EAV	Any number of fields	Many rows per response, difficult typing and complex reconstruction.
Single JSON blob for everything	Maximum flexibility	Would weaken stable patient/visit relationships and constraints.
Chosen hybrid	Relational template/visit links + JSON answers	Flexible answers while identity, version and visit remain strongly relational.

Viva sentence

We did not replace the relational model with JSON. We used JSON only at the variable boundary, while stable clinical entities remain normalized and linked by foreign keys.

06 Dynamic form architecture

TABLE	ONE ROW MEANS	IMPORTANT COLUMNS
custom_forms	One template version	form_key, name, version, draft/active/retired, creator
custom_form_fields	Fields in that version	key, label, type, options JSON, validation JSON, order
custom_form_responses	Current answers for one visit/version	answers JSON object, extra_fields JSON, revision version
custom_form_response_history	Immutable snapshots	response id, version, old answers, who changed, when

Lifecycle

```
create draft -> add/edit fields -> publish active
change required -> clone as new draft -> edit -> publish -> old active becomes retired
visit A response and visit B response remain separate because UNIQUE(visit_id, form_id)
```

Why version templates?

If a hospital changes a question next month, old answers must still point to the exact old definition. Editing a published template would change the meaning of historical data. Therefore published versions are treated as immutable and changes start in a new draft.

JSON checks

```
CHECK (JSON_TYPE(answers) = 'OBJECT')
CHECK (options IS NULL OR JSON_TYPE(options) = 'ARRAY')
CHECK (validation_rules IS NULL OR JSON_TYPE(validation_rules) = 'OBJECT')
```

Example JSON report expression

```
SELECT answers->>'$.night_symptoms' AS night_symptoms,
       CAST(answers->>'$.reliever_days' AS UNSIGNED) AS reliever_days
FROM custom_form_responses
WHERE deleted_at IS NULL;
```

07 Referential actions, soft deletion and audit

METHOD	WHERE	WHY
ON DELETE RESTRICT	Clinical/history relationships	Prevents deleting a visit or response when retained history depends on it.
ON DELETE CASCADE	Only replaceable configuration children	Draft form fields may disappear when their unpublished draft template is deleted.
status='retired'	Published form templates	Preserves old template versions without offering them for new use.
Soft delete	Patients and clinical records	Sets deleted_at, deleted_by and deletion_reason; data remains auditable/recoverable.

Why soft delete is safer in healthcare

- Clinical data may be legally or medically important later.
- Accidental deletion can be investigated and potentially restored.
- The reason and actor create accountability.
- Queries must include deleted_at IS NULL to hide inactive records.

Why only three cascades remain?

Two are role-permission configuration links and one is draft form fields. A database trigger blocks hard deletion of active or retired forms, so the field cascade is limited to replaceable unpublished configuration.

Audit log versus response history

audit_logs is a cross-system event trail: actor, action, entity and before/after data. custom_form_response_history is domain history: a complete answer snapshot per response revision. They overlap intentionally but answer different questions.

08 Indexes and query performance

INDEX TYPE	EXAMPLE	BENEFIT
Primary index	Created for every PK	Fast lookup by patient_id, visit_id, etc.
Unique index	medical_record_number; form_key+version	Enforces uniqueness and speeds lookup.
Foreign-key/supporting index	visits(patient_id, visit_started_at)	Speeds patient timeline and joins.
Composite index	custom_form_fields(form_id, section_order, display_order)	Matches filtering and ordering together.
Status index	custom_forms(status, form_key)	Speeds active/draft form discovery.

Composite index rule

A composite index is ordered left to right. An index on (form_id, section_order, display_order) helps queries starting with form_id. It is less useful for a query filtering only display_order.

Why not index every column?

Indexes consume disk space and every INSERT/UPDATE/DELETE must maintain them. Choose indexes from real WHERE, JOIN and ORDER BY patterns. Low-selectivity fields such as a boolean alone are often poor indexes.

EXPLAIN ANALYZE

```
EXPLAIN ANALYZE
SELECT * FROM visits
WHERE patient_id = 1 AND deleted_at IS NULL
ORDER BY visit_started_at DESC LIMIT 25;
```

What to say

EXPLAIN shows the optimizer plan; EXPLAIN ANALYZE also executes the query and reports actual rows and timing. I compare estimated versus actual rows, index usage, scan type, sorting and total cost/time.

09 Transactions, ACID, triggers, views and procedure

ACID	MEANING	PROJECT CONNECTION
Atomicity	All statements succeed or all roll back	Creating a visit and its record should not leave half a workflow.
Consistency	Constraints remain true	A response cannot reference a nonexistent visit.
Isolation	Concurrent work does not corrupt results	FOR UPDATE protects version creation against races.
Durability	Committed data survives failure	MySQL/InnoDB persists committed clinical updates.

Transaction example

```
START TRANSACTION;
INSERT INTO appointments (...) VALUES (...);
SET @new_id = LAST_INSERT_ID();
INSERT INTO audit_logs (...) VALUES (..., @new_id, ...);
COMMIT;
```

Triggers

Triggers automatically audit form changes and block deletion of published/retired templates. Advantage: protection applies regardless of client. Trade-off: hidden logic can surprise developers, so triggers must be documented and small.

Views

v_patient_visit_summary hides repeated joins for patient, visit, doctor and diagnoses. v_overdue_follow_ups centralizes the overdue rule and excludes soft-deleted data. A view stores a query, not a duplicate copy of rows.

Stored procedure

sp_patient_timeline(patient_id) returns visits in newest-first order. It demonstrates reusable parameterized database logic. The API can still use direct prepared queries where that is clearer.

10 Important queries - joins and reports

Explain each query as: source tables, relationship, filter, calculation, final ordering.

A. Patient visit timeline

```
SELECT * FROM v_patient_visit_summary
WHERE patient_id = 1
ORDER BY visit_started_at DESC;
```

Uses a view to give a longitudinal record. DESC puts the newest clinical event first.

B. Active allergies

```
SELECT p.medical_record_number, CONCAT(p.first_name,' ',p.last_name) patient,
       a.allergen, a.reaction, a.severity
FROM allergies a JOIN patients p USING(patient_id)
WHERE a.status='active' AND a.deleted_at IS NULL AND p.deleted_at IS NULL
ORDER BY a.severity DESC;
```

INNER JOIN returns only allergies with a valid patient. WHERE applies both clinical status and soft-delete visibility.

C. Abnormal laboratory results

```
SELECT p.medical_record_number, t.test_name, r.result_value, t.unit
FROM lab_results r
JOIN lab_orders o USING(lab_order_id)
JOIN lab_tests t USING(lab_test_id)
JOIN visits v USING(visit_id)
JOIN patients p USING(patient_id)
WHERE r.is_abnormal AND r.deleted_at IS NULL AND o.deleted_at IS NULL;
```

Follows foreign keys from result to order, test definition, visit and patient.

D. Prescription detail

```
SELECT pr.prescribed_at, m.generic_name, m.strength, pi.dose, pi.frequency
FROM prescriptions pr
JOIN prescription_items pi USING(prescription_id)
JOIN medications m USING(medication_id)
WHERE pr.patient_id=3 AND pr.deleted_at IS NULL AND pi.deleted_at IS NULL;
```

11 Important queries - analytics

E. Medication frequency (GROUP BY)

```
SELECT m.generic_name, m.strength, COUNT(*) AS times_prescribed
FROM prescription_items pi JOIN medications m USING(medication_id)
WHERE pi.deleted_at IS NULL
GROUP BY m.medication_id, m.generic_name, m.strength
ORDER BY times_prescribed DESC;
```

GROUP BY creates one group per medicine; COUNT(*) counts item rows in each group.

F. Latest vital signs (window function)

```
SELECT * FROM (
  SELECT p.medical_record_number, vs.*,
         ROW_NUMBER() OVER(PARTITION BY v.patient_id
                           ORDER BY vs.measured_at DESC) AS rn
  FROM vital_signs vs
  JOIN visits v USING(visit_id) JOIN patients p USING(patient_id)
  WHERE vs.deleted_at IS NULL
) x WHERE rn=1;
```

ROW_NUMBER restarts for each patient. Newest measurement receives rn=1. Unlike GROUP BY, the complete chosen row is retained.

G. Patients with above-average visit count (correlated comparison)

```
SELECT p.patient_id, p.medical_record_number, COUNT(v.visit_id) visit_count
FROM patients p JOIN visits v USING(patient_id)
WHERE p.deleted_at IS NULL AND v.deleted_at IS NULL
GROUP BY p.patient_id, p.medical_record_number
HAVING COUNT(v.visit_id) > (
  SELECT COUNT(*) / NULLIF(COUNT(DISTINCT patient_id),0)
  FROM visits WHERE deleted_at IS NULL
);
```

WHERE filters rows before grouping. HAVING filters aggregate groups. NULLIF prevents division by zero.

H. Patients without visits (LEFT JOIN anti-join)

```
SELECT p.*
FROM patients p
LEFT JOIN visits v ON v.patient_id=p.patient_id AND v.deleted_at IS NULL
WHERE p.deleted_at IS NULL AND v.visit_id IS NULL;
```

LEFT JOIN retains every patient. A null visit key identifies patients with no active matching visit.

12 SQL details examiners often ask

TOPIC	EASY ANSWER
WHERE vs HAVING	WHERE filters rows before grouping; HAVING filters grouped/aggregate results.
INNER vs LEFT JOIN	INNER needs a match; LEFT preserves all left rows and fills unmatched right columns with NULL.
UNION vs JOIN	UNION stacks compatible result rows; JOIN combines columns using a relationship.
COUNT(*) vs COUNT(column)	COUNT(*) counts rows; COUNT(column) ignores NULL values.
DELETE vs soft delete	DELETE removes the row; soft delete updates retention metadata and queries hide it.
DROP vs TRUNCATE vs DELETE	DROP removes object; TRUNCATE quickly removes all rows; DELETE removes selected rows and can be transactional.
CHAR vs VARCHAR	CHAR is fixed length; VARCHAR uses variable length and suits names/codes.
DATE vs DATETIME	DATE stores day only; DATETIME stores date and time.
Prepared statement	SQL structure and values are sent separately, reducing injection risk and improving reuse.
NULL	Means unknown/not supplied, not zero or empty string; compare with IS NULL.

Logical SELECT order

```
FROM / JOIN -> WHERE -> GROUP BY -> HAVING -> SELECT -> ORDER BY -> LIMIT
```

This explains why an aggregate condition normally belongs in HAVING and why an alias may work in ORDER BY but not always in WHERE.

Keyset pagination

```
SELECT visit_id, patient_id, visit_started_at
FROM visits
WHERE deleted_at IS NULL
  AND (visit_started_at, visit_id) < ('2026-09-22 09:31:00', 3)
ORDER BY visit_started_at DESC, visit_id DESC LIMIT 50;
```

Keyset pagination continues after the last seen ordered key. It is more stable and often faster than a very large OFFSET.

13 Database setup, migration and testing

STEP	ACTION	EXPLANATION
1	Fresh install	Run 001, then 002, seed 001, migration 003, and seed 002.
2	Existing old database	Back up first; run 003 if needed, then 004 for JSON conversion, 005 for retention, then follow-up seed only for demo.
3	Verification	Check row counts, foreign-key rules, trigger presence and JSON response reconstruction.
4	Performance	Run representative queries with EXPLAIN ANALYZE after realistic data volume.
5	Safety	Use synthetic data for demonstration; do not copy real hospital data into a student environment.

Why migrations are numbered

Database structure changes over time. Numbered migrations record order and intent so every environment can reach the same schema. Migration 004 is deliberately one-time because it transforms legacy tables; fresh installs already contain the final form architecture.

Useful verification SQL

```
SELECT COUNT(*) FROM patients;
SELECT COUNT(*) FROM doctors;

SELECT TABLE_NAME
FROM information_schema.COLUMNS
WHERE TABLE_SCHEMA='pediatric_clinic' AND COLUMN_NAME='deleted_at';

SELECT CONSTRAINT_NAME, DELETE_RULE
FROM information_schema.REFERENTIAL_CONSTRAINTS
WHERE CONSTRAINT_SCHEMA='pediatric_clinic';
```

14 Viva questions and model answers - core

Q. What is the main aim of your database?

A. To maintain connected pediatric patient, visit and clinical data with integrity, auditability, flexible forms and useful reports.

Q. Why MySQL 8?

A. It provides InnoDB transactions, foreign keys, checks, JSON, window functions, views, procedures and EXPLAIN ANALYZE in one widely used DBMS.

Q. What is database-first?

A. The schema, constraints and data rules are designed as the foundation; API/UI use them rather than replacing integrity with application-only checks.

Q. What is a primary key?

A. A unique non-null row identifier. Example: patient_id.

Q. What is a foreign key?

A. A constraint requiring a matching parent key. visits.patient_id ensures every visit belongs to an existing patient.

Q. Why AUTO_INCREMENT?

A. It generates simple surrogate IDs and avoids manual key collisions.

Q. What is referential integrity?

A. Relationships remain valid: no orphan visit, form response or prescription item can reference a missing parent.

Q. Why use a junction table?

A. patient_guardians resolves a many-to-many relationship and stores facts about the relationship itself.

Q. What is normalization?

A. Organizing data to reduce duplication and anomalies. Stable project tables generally follow 3NF.

Q. Give an update anomaly example.

A. If medication names were copied into every prescription item, renaming one medicine could leave inconsistent spellings.

Q. What is a composite key/index?

A. It uses multiple columns. UNIQUE(form_key, version) prevents two copies of the same template version.

Q. Why use constraints?

A. They enforce data quality for every application and import path, not only the UI.

15 Viva questions and model answers - design

Q. Why JSON for form answers?

A. Form fields change by template. JSON stores one typed answer document while relational keys still enforce form and visit identity.

Q. Why not store all clinical data in JSON?

A. Patients, visits, medicines and diagnoses are stable shared entities that need strong constraints, joins and consistent reporting.

Q. Why not use EAV?

A. EAV creates many rows per response, complicates typing, validation and reconstruction. The JSON document is simpler for one complete response.

Q. How do you protect historical form meaning?

A. Each template version has its own form_id; published versions are not edited, and old responses keep that exact foreign key.

Q. Why soft delete?

A. Clinical records should remain recoverable and auditable. We record deletion timestamp, user and reason instead of destroying the row.

Q. When is CASCADE allowed?

A. Only for replaceable configuration children such as fields of a deletable draft template and role-permission mappings.

Q. Why RESTRICT history deletion?

A. Deleting a parent must not silently erase clinical/audit evidence.

Q. What is an audit trail?

A. A record of actor, action, entity, before/after data and time. It supports accountability and investigation.

Q. Audit log versus soft-delete columns?

A. Soft-delete columns describe current deletion state; audit log records the deletion event and other changes over time.

Q. What is a trigger?

A. Database code that runs automatically on an event. We audit form changes and block deletion of published templates.

Q. What is a view?

A. A named saved query. It simplifies repeated joins and consistently applies visibility rules.

Q. What is a stored procedure?

A. Named parameterized database logic; sp_patient_timeline returns a patient's ordered visit history.

16 Viva questions and model answers - SQL/performance

Q. What is an index?

A. An additional ordered structure that speeds reads on selected columns but costs storage and write maintenance.

Q. How did you choose indexes?

A. From common foreign-key joins, patient timelines, form ordering, statuses and report filters - not by indexing every column.

Q. What is index selectivity?

A. How well a value narrows rows. A unique MRN is highly selective; a boolean alone is low-selectivity.

Q. What does EXPLAIN ANALYZE do?

A. Runs the query and reports the actual execution plan, row counts and timings for comparison with estimates.

Q. What is a transaction?

A. A unit of work committed together or rolled back together.

Q. Explain ACID.

A. Atomicity all-or-nothing; Consistency preserves rules; Isolation controls concurrency; Durability preserves committed data.

Q. Why use FOR UPDATE?

A. It locks selected rows during a transaction so two requests cannot create the same next version concurrently.

Q. WHERE versus HAVING?

A. WHERE filters source rows; HAVING filters grouped results after aggregation.

Q. What does GROUP BY do?

A. Creates groups with the same selected key so aggregates such as COUNT can be calculated per group.

Q. Why use ROW_NUMBER?

A. To rank rows inside each patient partition and select the latest complete vital-sign row.

Q. Why LEFT JOIN for patients without visits?

A. It preserves every patient; unmatched visits become NULL and can be selected.

Q. What is SQL injection and prevention?

A. Malicious input changes SQL structure. Use parameterized queries, validation and least privilege; never concatenate user values.

Q. Why keyset instead of OFFSET pagination?

A. Keyset uses the last ordered key, avoiding scanning/skipping a huge prefix and reducing page drift during inserts.

17 Tough viva questions and honest answers

QUESTION	STRONG ANSWER
Is this ready for a real hospital?	No. It is a strong academic prototype. Production needs SSO/RBAC enforcement, encryption/key management, backups, monitoring, privacy/legal review, clinical validation and tested FHIR/HL7 integration.
Why ENUM; is it always ideal?	ENUM gives strong control for small stable states. If states require frequent independent administration, a lookup table would be more flexible.
Does soft delete solve legal retention completely?	No. It is one mechanism. A hospital still needs formal retention, correction, access, archival and purge policies approved for its jurisdiction.
Can JSON be indexed?	Yes, commonly through generated columns or functional indexes for frequently queried paths. We keep current reporting simple and would add such indexes after measuring real workload.
What is a weakness of using user_id=1 in the prototype API?	It demonstrates auditing but is not real authentication. Production must take the authenticated identity from SSO and enforce authorization.
How would you integrate an existing hospital database?	Use an approved interface layer, usually FHIR/HL7 or vendor APIs, map external identifiers, begin read-only with synthetic/shadow data, reconcile differences and avoid direct writes to vendor tables.
How do you prevent duplicate patients?	MRN and external identifiers have uniqueness constraints, but real deployment also needs master-patient-index matching and a human reconciliation workflow.
What happens if the database update fails midway?	The transaction rolls back, preventing partial clinical workflows. Errors are returned to the caller; production would also monitor and retry appropriate operations.

Do not overclaim

A good viva answer separates what is implemented from what production would require. Saying 'prototype with verified database rules' is stronger than claiming hospital readiness without security and governance evidence.

18 Last-minute revision sheet

- Core path: patient -> visit -> record/diagnosis/vitals/prescription/lab/follow-up/form response.
- PK identifies; FK relates; UNIQUE prevents duplicates; CHECK validates a rule; INDEX speeds access.
- 1:M foreign key is on many side. M:N needs junction table.
- 3NF avoids duplication and update anomalies. JSON is used only for variable form content.
- Four form tables: template, fields, current response, response history.
- Template statuses: draft -> active -> retired. Published forms are versioned, not edited.
- Clinical deletion is soft: deleted_at + deleted_by + deletion_reason.
- RESTRICT protects retained history; CASCADE is limited to replaceable configuration.
- Transaction = all-or-nothing. ACID = Atomicity, Consistency, Isolation, Durability.
- WHERE before grouping; HAVING after grouping. LEFT JOIN can find missing relationships.
- ROW_NUMBER finds latest row per patient. GROUP BY creates aggregate groups.
- EXPLAIN ANALYZE verifies actual plan/timing. Indexes improve reads but slow writes.
- Prepared queries prevent SQL injection. API validation plus DB constraints gives defense in depth.

Best demonstration order

1. Show the patient list and open one record. 2. Explain patients and visits keys. 3. Show prescriptions/labs/follow-ups as related tables. 4. Open Clinical Forms and change the selected visit to prove visit-specific JSON responses. 5. Show response revision history. 6. Open the SQL files and explain one join, one GROUP BY, one window function and the transaction. 7. Finish with soft deletion and retention rules.

Final 20-second conclusion

The project combines a normalized relational clinical core with carefully limited JSON flexibility. Integrity comes from keys and constraints, safety comes from transactions, restrictive retention and audit history, and performance comes from workload-based indexes and query analysis. The design is demonstrable with synthetic data and clearly identifies the extra controls required before hospital production use.

ACADEMIC SAFETY NOTE	All demonstration records are synthetic. This handbook explains the implemented database and does not claim regulatory approval or production hospital readiness.